

ASSIGNMENT 4 REPORT - MULTI-CLIENT CHAT SERVER USING TCP

1. OBJECTIVE

The objective is to implement a concurrent TCP chat application where multiple clients communicate through one server, validate protocol behavior with packet capture, and evaluate performance for 1, 2, and 3 simultaneous clients.

2. NETWORK TOPOLOGY

Mininet topology used: 'single,4'

- * 'h1': Chat server ('10.0.0.1')
- * 'h2': ClientA ('10.0.0.2')
- * 'h3': ClientB ('10.0.0.3')
- * 'h4': ClientC ('10.0.0.4')

Execution backend details from 'run_info.txt':

- * 'SWITCH_BACKEND=LinuxBridge (WSL default)'
- * 'PINGALL_LOSS_PERCENT=0.0'

3. PROGRAM DESIGN

SERVER ('SERVER.PY')

- * Listens on TCP port '5000'.
- * Accepts multiple clients concurrently using threads.
- * First line from each client is treated as username registration.
- * Broadcasts each received message to all connected clients in format:
 - '[username] message'
- * Logs connection lifecycle events in 'server_events.log' using:
 - 'timestamp,EVENT,username,client_ip'
- * Logs chat messages in 'chat_log.txt' using:
 - 'timestamp,username,message'

CLIENT ('CLIENT.PY')

- * Registers username immediately after connecting.
- * Supports interactive mode and auto mode ('--auto-count', '--auto-delay').
- * Measures own message broadcast round-trip delay samples.

4. EXPERIMENTAL RESULTS

Performance experiment executed for 1, 2, and 3 clients. Each client sent 20 messages.

From 'performance_results.csv':

clients	total_messages	avg_delivery_time_ms	throughput_msgs_per_sec
1	20	1.619	11.987
2	40	28.768	23.947
3	60	29.007	35.880

Observations:

- * Total delivered messages scale linearly with client count.
- * Throughput increases with more clients due higher aggregate send rate.
- * Average delivery time rises under concurrent multi-client load.

5. WIRESHARK VERIFICATION

Capture filter used: 'tcp.port == 5000'.

Captured artifact: 'capture.pcapng'

Required screenshot files:

- * 'screenshots/tcp_handshake.png'
- * 'screenshots/chat_message.png'
- * 'screenshots/broadcast_message.png'
- * 'screenshots/connection_close.png'

Verified protocol events:

- * TCP three-way handshake ('SYN', 'SYN-ACK', 'ACK').
- * Chat message packet with payload ('PSH,ACK') from client to server.
- * Server payload broadcasts to multiple client destinations.
- * TCP connection teardown ('FIN/ACK' sequence).

6. GRAPH ANALYSIS

Generated graphs:

- * 'graphs/clients_vs_delay.png'
- * 'graphs/clients_vs_throughput.png'

Trend summary:

- * Delay graph shows low latency for one client and higher delay when multiple clients send simultaneously.
- * Throughput graph rises from 1 to 3 clients, indicating increased aggregate message handling capacity.

7. REFLECTION ANSWERS (<= 300 WORDS)

A multi-threaded server is preferred because each client can be handled without blocking others. In a single-threaded design, one slow client operation can delay all remaining clients and reduce responsiveness.

When many clients communicate at the same time, common challenges include shared-state synchronization, ordering of broadcasts, client disconnect handling, and avoiding race conditions in message/event logs.

TCP reliability comes from connection-oriented state, sequence numbers, acknowledgments, retransmissions, and in-order delivery guarantees. For chat, this means messages are delivered reliably and without corruption at the application layer.

Wireshark verification was supported by observing handshake packets, application-data packets ('PSH,ACK'), server-to-client broadcast packets, and connection termination ('FIN,ACK'). These packet-level observations matched the server/client runtime behavior and log timelines.

8. CONCLUSION

The assignment requirements were implemented with a threaded TCP server, username registration, broadcast messaging, performance measurement for 1/2/3 clients, packet-capture verification, and graph generation. The collected logs, CSV, graphs, and packet evidence are consistent with expected multi-client chat behavior.